

at the AMX issue port, not memory-bound.

`dispatcher-m-sweep.md`

M-sweep on GR and Zen 4, and the argument for why the M=4 and M=8 bucket boundaries sit where they do.

amd-blocked-avx-crossover.md	Zen 4 / Zen 5 blocked-AVX vs dotter crossover, kernel level.
blocked-avx-amd-DECISIONS.md	The AMD spike decision log (D1–D6), the –6.6% full-stack result, and the recipe for the gpt-oss Zen 4 measurement that never completed.
amx-llama-summary.md	First AMX-vs-AVX kernel numbers. Source of the "AMX P.V is free" finding — V's existing layout is already byte-identical to the AMX B-tile.
llama-attention-summary.md	Per-slice benchmarks at M=16 and M=4, with and without pre-packed K.
runtron-llama-3-8b.md	End-to-end runtron baseline vs delphi, plus the original integration pseudocode.
comparison.md	Delphi preset vs baseline compiler-tuning deltas across <code>t_dotter</code> / <code>t_wide</code> . Tuning alone is ~0% on this workload.

Also untracked and worth reading: [fixes.txt](#) — a running log of review feedback on the branch and how each item was resolved. It records, among other things, that the top-level `gpt_oss*.sh` runners are deliberately kept out of the PR.

01 Orientation

Start at **Note [K VNNI mirror]**, [h/tron/kernels/attention_dispatch.hpp:24](#). It is the design document for the whole subsystem: storage, writer, readers, and the exact predicate that gates them. Everything below assumes you have read it.

The branch adds ~3,587 lines across 40 files. These are the ones that matter.

FILE	ROLE	LINES
<code>kernels/amx_kv_layout.hpp</code>	The format. <code>scatter_k_vnni_token()</code> is its defining formula; pack/unpack helpers around it.	152
<code>kernels/attention_dispatch.hpp</code>	Note [K VNNI mirror], the eligibility predicates, range-mask and softmax-pack helpers.	200
<code>kernels/sw_attention.hpp</code>	M-bucketed dispatch: <code>qk / pv / scalar_pv</code> .	187
<code>kernels/amx_attention.hpp</code>	AMX <code>tdpbf16ps</code> kernels.	118
<code>kernels/avx512_blocked_attention.hpp</code>	Blocked AVX-512 kernels, <code>vdpbf16ps</code> , broadcast-Q × VNNI-K.	133
<code>kernels/attn_profile.hpp</code>	Instrumentation. Thread-local by necessity — see the 80% trap below.	281
<code>simd/amx.hpp</code>	<code>arch_prctl</code> permission request, thread-local tile config, <code>is_supported()</code> .	105
<code>models/kv_cache.hpp</code>	The mirror's storage and accessors. Primary edit target.	+82
<code>models/self_attention.hpp</code>	Call sites: M-stacking, mask-after-matmul, batched 3-pass. Where the risk lives.	+423
<code>models/model.hpp</code>	<code>save_k</code> — the mirror's only writer.	+16
<code>src/amx_microbench.cpp</code>	Standalone perf binary. Not a Catch2 test; lives in <code>src/</code> on purpose.	274
<code>t/t_amx_attention.cpp</code> · <code>t_sw_attention.cpp</code>	Kernel + dispatcher tests. 19,458 and 15,552 assertions.	707

Build gating

Everything is behind `SW_ATTN_DISPATCH` , which is set only when `DELPHI` is on (`CMakeLists.txt:97`) — the sole

Branch topology

- **bill-amx** → PR #2934, open, **not** merged to main. Nine commits from `5e90e3ea6` to `309168e2e` .

preset that adds the `-march` flags defining `__AMX_BF16__`. On any other build the whole subsystem compiles out.

Consequence for you: CI cannot exercise these tests until [#2999](#) lands. Build locally with `cmake --preset delphi`.

- **hrv-amx-ci-enablement** → PR #2999, open. Hannah Van Eenoo's, built *on top of* bill-amx (merged it in 2026-06-17). Delphi CI enablement plus two dispatcher gates.
- Merge-base with main is `8811bdd35`.

02 Why the format exists

One packing serves three consumers. `K_vnni` is byte-identical to the AMX B-operand for `tdpbf16ps`; the blocked-AVX kernel reads the *same bytes* via `vdpbf16ps`; and V needs no packing at all, because `set_v` already emits the pair-interleaved form both P·V kernels want. That three-way alignment is the load-bearing fact of the design.

VNNI = Vector Neural Network Instructions. In the bf16 flavor, each 32-bit lane holds a pair of adjacent contraction-dimension values, so one instruction does two multiply-accumulates per lane into an fp32 accumulator. The "VNNI layout" is the operand packing that requires: K-pairs interleaved as {even, odd} within each lane.

Dispatcher M-sweep, one (kv_head, page) slice at llama-3-8B shape — head_dim 128, page_size 64. MEASURED

M	GR DOTTER	GR DISPATCH	GR WIN	ZEN 4 DOTTER	ZEN 4 DISPATCH	ZEN 4 WIN	KERNEL TAKEN
1	628 ns	720 ns	0.87×	261 ns	502 ns	0.52×	scalar dotter
2	1,255 ns	1,445 ns	0.87×	508 ns	960 ns	0.53×	scalar dotter
4	2,508 ns	1,644 ns	1.53×	1,012 ns	860 ns	1.18×	blocked AVX-512
8	5,020 ns	1,551 ns	1.14×	2,018 ns	1,500 ns	1.17×	AMX / blocked

	8	5,020 ns	451 ns	11.1×	Single-Format VNNI KV	2,018 ns	1,723 ns	1.17×	AMX / blocked
	16	10,036 ns	479 ns	21.0×		3,972 ns	3,438 ns	1.16×	AMX / blocked

Where it pays end-to-end MEASURED

gpt-oss-120b-tp4, 4 users, Granite Rapids:

CONTEXT	BASE	AMX	Δ
2K / 1K	631.89	648.74	+2.7%
8K / 1K	514.92	670.35	+30.2%

Llama-3-8B is unchanged. **gpt-oss is the model that justifies the work** — longer context means more visible keys per page and more traffic through the batched path. PR #2934's title cites +11.7% TPS as its headline aggregate.

Where it does not MEASURED

llama-3-8B on Zen 4 (EPYC 9654, andoria-15), 2K/1K, 4 users, 8 FPGAs, three interleaved runs: **724.0 → 676.2 tok/s, -6.6%**.

The 1.20× kernel win does not survive the full stack. Diagnosed as: the mirror write in `save_k` is pure cost with no AMX tier to offset it; +50% cache pressure; `apply_page_tok` computes the full $M \times \text{page_size}$ S then samples visible columns where the dotter computes only visible keys; and `softmax_pv` — the largest single component at 87s — is untouched.

This is the cautionary result. It is the same mechanism the task below has to avoid re-creating.

03 The current design, and what it costs

K is stored *twice*. Canonical row-major k lives at `kv_cache.hpp:941`; the VNNI mirror `k_vnni` sits beside it at `:963`, written through by `set_k_vnni` on every saved token. Three costs follow:

- **+50% kv_block**. This is DMA memory (`make_unique_dma_for_overwrite`), so it is the expensive kind. It has already bitten: hrv's `d94e67e5e` had to bound the TTV2 cross-branch test cache footprint explicitly "when K VNNI mirrors are compiled in"

STAND TEST CACHE FOOTPRINT EXPLICITLY WHEN K VNNI MIRRORS ARE COMPILED IN.

- **Over-allocation.** `has_k_vnni_mirror` is head-size-only because `kv_block` is not templated on `kv_mul`. So llama-3.2-3B (`kv_mul=3`, head 128) allocates a mirror it never reads. Flagged [REVISIT] in the Note.
- **A correctness hazard.** Writer and reader gates must agree *exactly*; a reader firing where the writer did not reads zeros and silently corrupts scores. That single sentence is why ~40 lines of Note exist.

The dual format is visible right in the interface — `sw_attention::qk(Q, K_data, K_vnni, ...)` takes both pointers and picks internally (`self_attention.hpp:1206, :1369`).

The compile-time predicate, single source of truth for storage, writer and both dispatch entry points:

```
constexpr bool vnni_eligible(size_t head_dim, size_t kv_mul) {
    return mirror_eligible(head_dim) && kv_mul >= 4;    // head_dim ∈ {64,128}
}
```

Format is keyed on `kv_mul`, never on runtime M: K is written once per token but read by an unknown number of queries later, so the stored format must serve every future read. `kv_mul=3` is the one shape whose reads straddle the scalar/blocked boundary (M=3 vs 6), so llama-3.2-3B is kept wholly scalar.

04 The task

Make VNNI the single format. Delete the mirror; store K only in VNNI; keep a contiguous form alive solely where cost is acceptable — the scalar dotter path and unit tests.

The precedent already exists, ten lines from the K code

V is already single-format. Under `TRON_CHUNK_SIZE == 16`, V has no contiguous copy and no view accessor at all — only `get_v(layer, kv_head, i_page, bf16* out)`, which unpacks into a caller buffer. "One non-obvious storage format, unpack at the boundary" is an established pattern in this file, not a novel risk.

Every reader of contiguous K

Complete inventory, verified by `grep` across `h/ src/ t/`.

SITE	WHAT IT NEEDS	UNDER SINGLE-FORMAT VNNI
<code>kv_cache.hpp:982</code> <code>k_at()</code>	—	Zero callers. Already dead. Delete it.
<code>model.hpp:1864</code> <code>save_k</code>	writer	Already writes VNNI via <code>scatter_k_vnni_token</code> . Drop the contiguous write.
<code>full.hpp:1982</code> <code>k_head_fn</code>	one token's row → FPGA	Teach the GOF shuffle to read VNNI. See §05 — this is the crux.
<code>kv_cache.hpp:722</code> page copy	bulk memcpy	Per-token scatter, as <code>append_v</code> already does for V. Copies aligned mod 16 can still memcpy whole tiles.
<code>self_attention.hpp:1270</code> dotter	row per key	Strided VNNI reads, or unpack. Acceptable — this is the scalar path.
<code>t_llama_unit.cpp</code> (×10)	rows	Keep an unpack accessor, test-only.

Implementation sequence**01 Write `shuffle_k_head_vnni` TO DO**

Sibling to `shuffle_k_head` / `shuffle_k_head_padded` in `h/pos/hwattention.hpp`. Same destination writes, source index becomes $(e \gg 1) * 32 + (e \& 1)$ off the token's tile base. **Do not** add a `get_k()` staging buffer — that inserts a copy that need not exist.

02 Change `page_info::k_head_fn` TO DO

`gof.hpp:62` returns `bf16*` — a pointer to a contiguous row that will no longer exist. It has to hand the shuffle a tile base plus token index instead. Touches the two GOF tests that fabricate their own K buffers: `t_gof_dma.cpp:342` and `t_gof_staging_leaks.cpp:124`.

03 Measure `gof_populate` both ways GATE

Do this before going further. A `TRACE_EVENT("hwattention", "gof_populate")` is already in place at `src/tron/gof.cpp:91`. See §06 for what could go wrong and why it probably will not.

04 Delete the mirror TO DO

`k_vnni[]` becomes `k[]`. Drop `k_contig_block`, `has_k_vnni_mirror`, `set_k_vnni`, and the `mirror_eligible` / `vnni_eligible` split. Collapse `sw_attention::qk` to one K pointer. Most of `mirror_runtime_enabled()` goes with it — there is no mirror left to gate.

05 Rewrite Note [K VNNI mirror] TO DO

Its central hazard — "writer and reader gates must agree exactly or you silently read zeros" — exists *only* because there are two formats. One format, no gate, no hazard. What survives is a much shorter note describing the layout and the `kv_mul-keyed`

dispatch decision.

06 Re-run the AMD measurement BLOCKED

The -6.6% llama regression was attributed in part to the mirror write and cache pressure. Removing both changes that calculus. The outstanding question — gpt-oss-120b on Zen 4 — is still unmeasured; see §07.

05 Why the GOF path is nearly free

The natural objection to single-format VNNI is that the hardware offload path needs contiguous rows, so you would pay an unpack per (token, layer, kv_head) — production cost, not test cost. **That objection is wrong**, and the reason is worth internalizing before you start.

The GOF path *already* reformats K element-by-element. `gof_populate` (`src/tron/gof.cpp:97`) calls `k.set_position()` → `shuffle_k_entry` → `shuffle_k_head`, which is a transpose into HBM word order, not a memcpy:

```
for (size_t col = 0; col < hbm_word_width; ++col) {
    k_hw_head[col] = k_head[4 * col];
    k_hw_head[hbm_word_width + col] = k_head[(4 * col) + 1];
    k_hw_head[(2 * hbm_word_width) + col] = k_head[(4 * col) + 2];
    k_hw_head[(3 * hbm_word_width) + col] = k_head[(4 * col) + 3];
}
```

It already reads the source stride-4. The padded variant — which is what gpt-oss's `head_size=64` takes — is fully element-wise: `k_hw_head[offset] = k_head[element]`. Element count is identical either way. **VNNI changes only the source index**

expression. A different sort, not an added copy.

Note the asymmetry this reveals: *V* *does* pay a staging copy today (`v_buf + get_v` at `full.hpp:1985`), purely because `shuffle_v_entry` was never taught the interleaved layout. *K* done properly will be cleaner than *V*, and *V* has a latent cleanup of its own there.

06 Risks, and traps already sprung

Cache-line footprint MEASURE

The one real residual. Contiguous: one token's 128 elements span 4 cache lines. VNNI: each 64-byte line holds one k-pair across 16 tokens, so one token's gather touches **64 lines using 4 bytes each**. A GOF is `gof_tokens = 4` positions sharing those same lines — 16 bytes used per line, ~64 lines vs ~16.

That is 4 KB, comfortably L1-resident, and `save_k` just wrote those exact lines. It should be noise — but "should be" is why step 3 measures instead of assuming.

It could also come out *better*: both layouts are permutations of the same 128 elements, and VNNI's consecutive pairs are friendlier to a vectorized shuffle than the current stride-4 scalar reads.

Traps already sprung LEARNED

- **Atomic profiling counters cost 80% of wall** on gpt-oss-120b through contention. `attn_profile` is thread-local + registry-walk-at-dump for that reason; it now sits within ~1% of uninstrumented. Do not "simplify" it back.
- **-j192 OOM-thrashes andoria.** Dual ingest-enabled libtron builds drove the box into memory thrash and made sshd unreachable. Build one at a time at `-j 48`.
- **Enabling head_dim=64 on the single-job path regressed gen mode.** `apply_page_tok`'s `use_dispatcher` is pinned to `head_size==128` deliberately: at `M=kv_mul=8` you waste half the AMX tile and per-call setup overwhelms the win. gpt-oss wins via the *batched* path at `M=16`.
- **M<4 regresses and that is accepted.** `scalar_pv` reads real pair-interleaved *V* where the dotter used a cheaper *V*-as-*K* proxy. Correct-not-fast, sub-microsecond. sparse residual regime.

Two gates from hrv's branch must survive any refactor — re-check them after step 4:

- **bf16 Q storage** (57e4fd5ed , restored in 6fe7f4d95): float-backed host Q tensors must not enter the bf16 dispatcher. It was dropped once already by a merge.
- **Determinism** (d94e67e5e): when `pay_for_determinism` is set, stay on the scalar reference path.

07 Open questions

QUESTION	STATE	NOTES
Does gpt-oss-120b win on Zen 4?	BLOCKED	The measurement that decides whether AMD enablement ever happens. Never completed — andoria memory thrash. <code>gen-ao-gpt-base/</code> and <code>gen-ao-gpt-disp/</code> in Bill's tree are the partial attempt. Recipe is in <code>blocked-avx-amd-DECISIONS.md</code> .
Can the FPGA consume VNNI K directly?	UNASKED	Would make VNNI the format end-to-end and delete <code>shuffle_k_head</code> entirely. FPGA-side change, out of scope for this PR, but worth raising with the hardware team.
Does <code>gof_populate_regress</code> ?	MEASURE	Step 3. The gate on the whole approach.
Should V follow K?	OPTIONAL	Teaching <code>shuffle_v_entry</code> the interleaved layout would delete V's <code>v_buf</code> staging copy. Independent cleanup; do not bundle it.

Suggested PR split

The branch is ~3,587 lines across 40 files, which is a hard single review. It splits cleanly on *new files* vs *edits to hot-path code* — the kernel tests include only kernel

headers and `amx_kv_layout.hpp`, never `kv_cache.hpp` or `self_attention.hpp`, so the kernel layer stands alone:

- **First:** #2999 (Delphi CI) — without it nothing else is testable in CI.
- **Then kernels + format** (~1,670 lines, all new files, zero behavior change): the five kernel headers, the microbench, both test files.
- **Then integration:** `self_attention.hpp` call sites, the storage change, `save_k`, the GOF shuffle. Single-format makes this review markedly easier — there is no +50% memory cost left to defend, and hrv's TTV2 workaround can be dropped with it.

08 Commands

```
# Build (Delphi is the only preset that compiles any of this)
cmake --preset delphi
cmake --build gen-delphi -j 48 --target amx_microbench t_amx_attention t_sw_attention

# Kernel + dispatcher tests
gen-delphi/t_amx_attention          # 19,458 assertions
gen-delphi/t_sw_attention           # 15,552 assertions

# Microbench: <M> <iters> <variant: v1|v2|avx|dotter|dispatch>
for m in 1 2 4 8 16; do
    taskset -c 4 gen-delphi/amx_microbench $m 500000 dispatch
    taskset -c 4 gen-delphi/amx_microbench $m 500000 dotter
done

# Roofline / topdown
sudo perf stat -e cycles,instructions,L1-dcache-loads,L1-dcache-load-misses,\
    l2_rqsts.all_demand_data_rd,LLC-loads -- \
    taskset -c 4 gen-delphi/amx_microbench 16 5000000 v2
```

```
# Slice discipline on a shared box – set your partition before any make test
SYSTEM_CONFIG='--instance 1,2' make test-host
```

Note the CLAUDE.md slice contract: after materially changing a test, refresh its bench entry with `bin/slice bench --update <test>` (or `--new`), or the packer mis-schedules it. And any PR touching CMake, the GNUmakefile, or `ingest/` needs the "**Continuous integration**" label on GitHub to get the full matrix.

09 Terms

VNNI	Vector Neural Network Instructions. Here the bf16 flavor, <code>vdpbf16ps</code> : a 32-bit lane holds a pair of adjacent contraction values, two MACs per lane per instruction. The layout is the operand packing it requires.
M / N / K	GEMM dims, $C[M,N] += A[M,K] @ B[K,N]$. For Q·KT: M = query rows = <code>n_visible × kv_mu1</code> ; N = keys = <code>page_size</code> ; K = <code>head_dim</code> . P·V swaps the latter two. Watch the collision — K is both the contraction dim and the key tensor.
kv_mu1	<code>n_heads / n_kv_heads</code> , the GQA ratio. Sets the floor on M and therefore the storage format. llama-3-8B is 4, gpt-oss is 8, llama-3.2-3B is 3.
GOF	The hardware-offload staging unit. <code>gof_tokens</code> = 4 token positions per cacheblock; host-side DMA staging in HBM layout, rebuilt from the host KV pages after eviction. See Note [GOF staging eviction].
M buckets	Dispatcher tiers: $M \geq 8 \rightarrow \text{AMX}$, $M=4-7 \rightarrow \text{blocked AVX-512}$, $M \leq 3 \rightarrow \text{scalar dotter}$, $M=0 \rightarrow \text{skip}$. <code>N_TILE=16</code> keys per ZMM accumulator, <code>M_MICRO=4</code> query rows in flight, AMX <code>M_TILE=16</code> — which is why the AMX bucket starts at half-full.